

LAB1_Report_ALU

컴퓨터소프트웨어학부 2021026108 김민제

ALU.cpp 구현

main.cpp 는 compute 함수를 통해서 제공된 100개의 testcase들을 확인해주고 랜덤한 testcase를 만들어주는 구조입니다. compute 함수를 구현하기 위해서 ALU.cpp 파일의 빈 부분들을 각 경우들에 따라 아래의 방식으로 완성시켰습니다.

1. ALU_ADDU

오버플로우를 계산하지 않고 add 연산을 하는 것이기에 그냥 단순 더하기 연산을 실행해서 32비트 unsigned int 에 넣어주기 때문에 오버플로우가 발생하더라도 무시하고 넣어줘 따로 고려하지 않아도 되었습니다.

```
*alu_result = operand1 + operand2;
```

2. ALU_AND

논리적 and 연산을 수행하는 것이기에 c에서 기본적으로 지원하는 &연산을 사용하여 구현했습니다.

```
*alu_result = operand1 & operand2;
```

3. ALU_NOR

or 연산을 수행하고 not 연산을 수행하는 것으로 구현했습니다.

```
*alu_result = ~(operand1 | operand2);
```

4. ALU_OR

논리적 or 연산을 수행하는 것이기에 c에서 기본적으로 지원하는 | 연산을 이용하여 구현했습니다.

```
*alu_result = operand1 | operand2;
```

5. ALU_SLL

left shift 연산을 수행하는 것이기에 << 연산자를 이용하여 구현했습니다.

```
*alu_result = operand2 << shamt;
```

6. ALU_SRA

arithmetic right shift 연산을 수행하는 것이기에 부호가 유지되어야하는데 입력받은 데이터는 unsigned 데이터여서 signed로 변환하고 shift 연산을 수행해줘야 구현이 가능했습니다.

```
*alu_result = (signed)operand2 >> (signed)shamt;
```

7. ALU_SRL

logical right shift 연산을 수행하는 것이므로 단순히 c의 >> 연산자를 이용해서 구현했습니다.

```
*alu_result = operand2 >> shamt;
```

8. ALU_SUBU

오버플로우를 무시하는 sub 연산이므로 add 연산과 동일하게 단순히 - 연산자를 이용해서 구현

했습니다.

```
*alu_result = operand1 - operand2;
```

9. ALU_XOR

xor 연산이므로 c의 ^ 연산자를 이용해서 구현했습니다.

```
*alu_result = operand1 ^ operand2;
```

10. ALU_SLT

부호가 있는 less 연산이므로 signed로 변환해주고 < 연산자를 이용해서 구현했습니다.

```
*alu_result = (signed)operand1 < (signed)operand2;
```

11. ALU_SLTU

부호가 없는 less 연산이므로 그냥 입력받은 operand 끼리 < 연산자를 이용해서 구현했습니다.

```
*alu_result = operand1 < operand2;
```

12. ALU_EQ

equal 연산이므로 == 연산자를 이용해서 구현했습니다.

```
*alu_result = operand1 == operand2;
```

13. ALU_NEQ

not equal 연산자이므로 != 연산자를 이용해서 구현했습니다.

```
*alu_result = operand1 != operand2;
```

14. ALU_LUI

upper bit에 immediate 값을 load 해주는 연산자 이므로 16bit left shift 해주는 걸로 구현했습니다.

```
*alu_result = operand2 << 16;
```

ALU.v 구현

verilog의 구현의 경우 cpp과 대부분 동일한 코드여서 구조가 다르거나 다른 연산자를 사용한 경우만 적었습니다.

ALU_tb.v에서 c++에서 생성한 랜덤 testcase를 확인해서 passed, failed 값을 변경시켜 코드가 올바른지 아닌지 확인할 수 있었습니다.

1. ALU_SRA

verilog에서는 >>> 연산자로 Arithmetic right shift 를 지원해서 처음에는 그냥 >>> 연산자만을 사용해서 구현했습니다. 계속 failed 되는 값이 생겨서 값을 확인해보니 signed bit가 유지되지 않고 있다는 것을 확인했습니다. 아마도 unsigned int로 해석하기 때문에 signed bit를 모르고 채우는 것이라고 생각을 해서 signed로 변환을 해주고 shift 연산을 하니 잘 동작했습니다.

verilog 연산에 대해서 더 공부해보니 operand가 unsigned 인 경우 부호를 고려하지 않아 내부적으로 >> 연산자처럼 동작해서 올바르게 arithmetic shift를 하려면 signed로 형 변환을 해주는 과정이 필요했습니다.

```
alu_result = $signed(operand2) >>> shamt;
```

테스트 결과 확인

c++ 코드에서 random으로 c++에서 구현한 alu 연산을 바탕으로 랜덤한 mips 명령어를 만들고 그 계산결과를 작성해주는 코드가 있어 1000개의 testcase를 만들어서 verilog 코드를 동작시켜 확인했다. 이때 rand() 함수는 항상 동일한 값을 줘서 srand(); 로 seed 값을 변경하여 총 10회 test 했습니다.

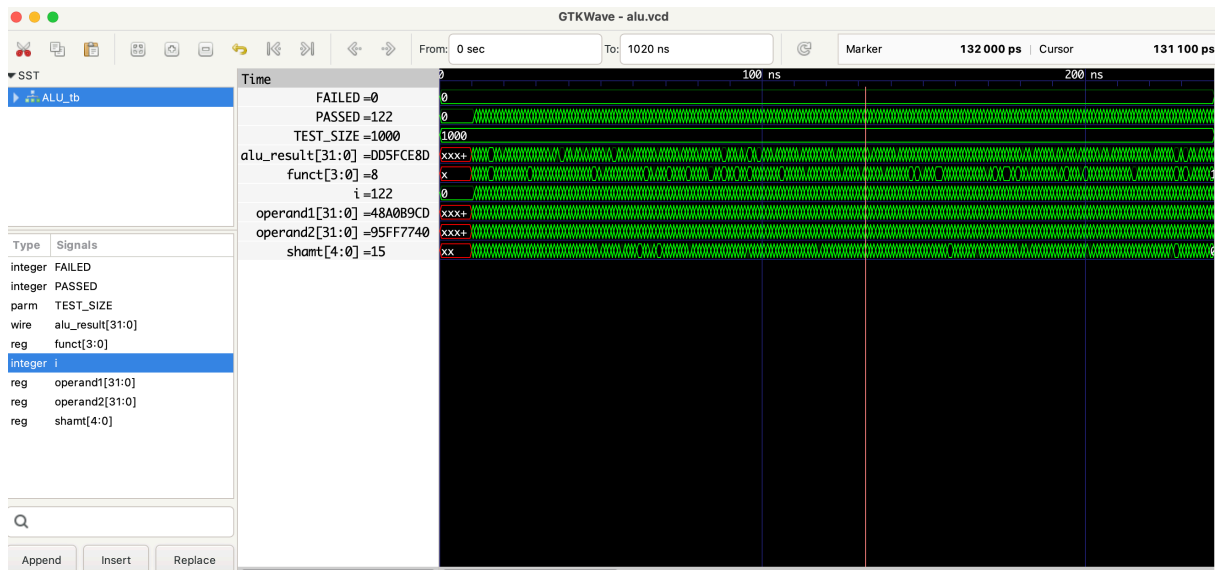
```
srand((unsigned int)time(NULL));
for (int cycle = 0; cycle < 1000; cycle++) {
    operand1 = ((rand() << 1) + rand());
    operand2 = ((rand() << 1) + rand());
    shamt = rand() % 32;
    aluop = rand() % 14;

    alu.compute(operand1, operand2, shamt, aluop, &alu_result);

    // Write down the file! (to use as testbench!)
    fout_op1 << std::setw(8) << std::setfill('0')
        << std::hex << operand1 << endl;
    fout_op2 << std::setw(8) << std::setfill('0')
        << std::hex << operand2 << endl;
    fout_shamt << std::bitset<5>(shamt) << endl;
    fout_funcnt << std::bitset<4>(aluop) << endl;
    fout_result << std::setw(8) << std::setfill('0')
        << std::hex << alu_result << endl;
}
```

10회의 테스트 결과 모두 잘 동작하는 것을 확인했고 테스트 케이스 중 하나의 실행 모습과 gtkwave를 통해 확인한 vcd파일의 사진입니다.

```
PASSED
TEST CTRL Sig: 8 :
PASSED
TEST CTRL Sig: a :
PASSED
TEST CTRL Sig: c :
PASSED
PASSED =          1000, FAILED =          0
ALU_tb.v:78: $finish called at 1020000 (1ps)
```



Trouble Shooting

아직 단순한 alu 를 구현하는 것이라 큰 문제는 없었지만 shift 연산을 구현하는 부분에 있어서 약간의 문제가 있었습니다. 교수님께서 주신 과제 명세서에서 shift 연산은 operand1을 대상으로 수행하는 것으로 표기가 되어있어 교수님이 제공해주신 testcase에서 문제가 발생하여 MIPS ISA를 확인해보니 shift 연산은 rs가 아니라 rt를 대상으로 해서 rs인 operand1이 아니라 rt인 operand2에 shift 연산을 수행해야 된다는 것을 알았습니다. 그래서 operand1이 아니라 operand2에 shift 연산을 수행하도록 코드를 수정한 결과 잘 동작하는 것을 확인할 수 있었습니다.